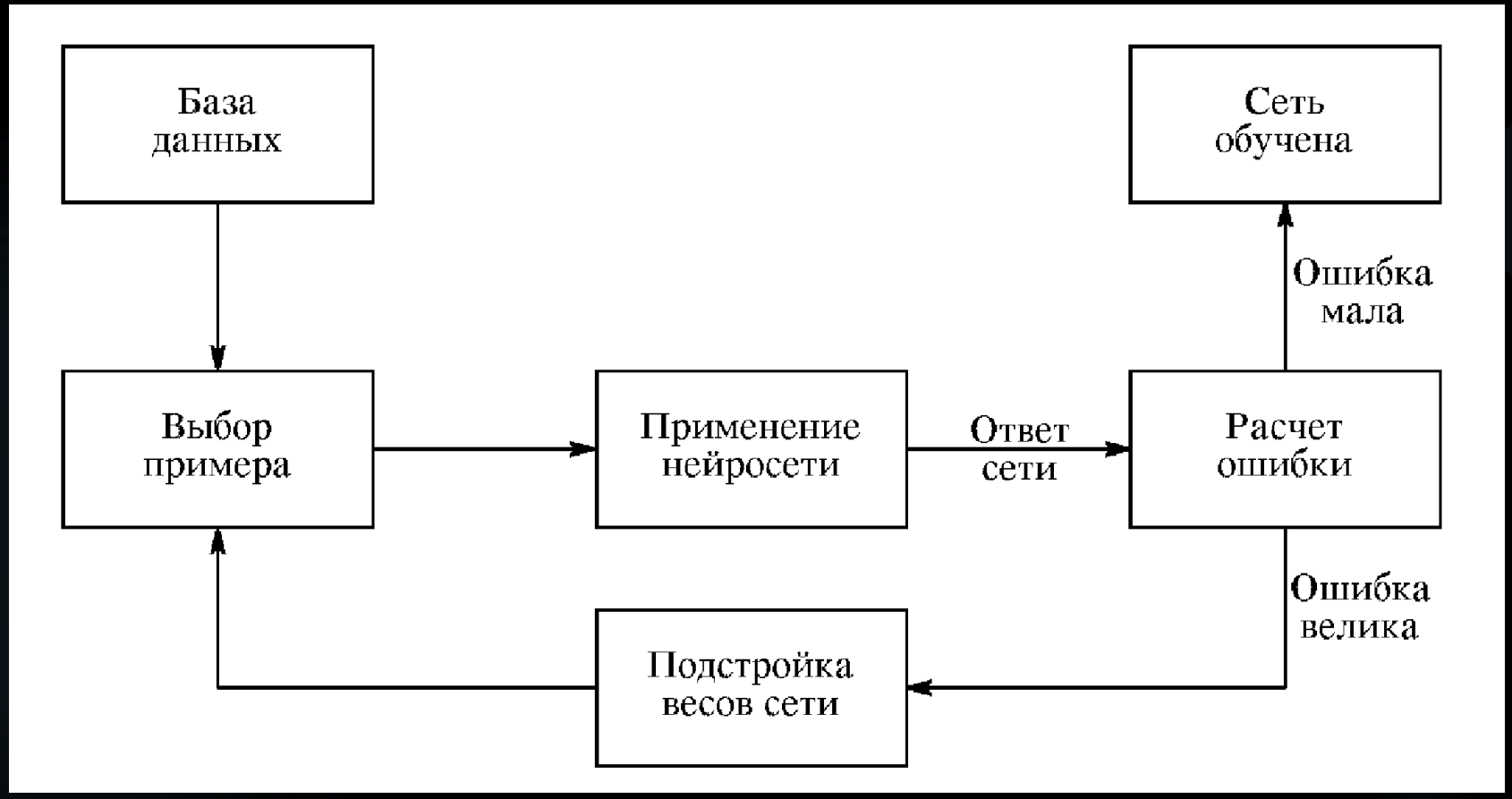
Обучение искусственных нейронных сетей

Введение

- Обучить нейросеть — значит, сообщить ей, чего мы от нее добиваемся. Этот процесс очень похож на обучение ребенка алфавиту. Показав ребенку изображение буквы «А», мы спрашиваем его: «Какая это буква?» Если ответ неверен, мы сообщаем ребенку тот ответ, который мы хотели бы от него получить: «Это буква А». Ребенок запоминает этот пример вместе с верным ответом, т.е. в его памяти происходят некоторые изменения в нужном направлении. Мы будем повторять процесс предъявления букв снова и снова до тех пор, когда все буквы будут твердо запомнены. Такой процесс называют «обучение с учителем»

Иллюстрация процесса обучения ИНС с учителем



Математическая постановка

- Математически процесс обучения можно описать следующим образом.
- В процессе функционирования нейронная сеть формирует выходной сигнал **Y** в соответствии с входным сигналом **X**, реализуя некоторую функцию **Y = G(X)**. Если архитектура сети задана, то вид функции **G** определяется значениями синаптических весов и смещений сети.
- Пусть решением некоторой задачи является функция **Y = F(X)**, заданная парами входных-выходных данных $(\mathbf{X^1}, \mathbf{Y^1})$, $(\mathbf{X^2}, \mathbf{Y^2})$, ..., $(\mathbf{X^N}, \mathbf{Y^N})$, для которых $\boldsymbol{Y^k = F(X^k)}$ $(k = 1, 2, \ldots N)$. Обучение состоит в поиске (синтезе) функции G, близкой к F в смысле некоторой функции ошибки *E*.

Математическая постановка

- Если выбраны множество обучающих примеров — пар $(\mathbf{X}^k, \mathbf{Y}^k)$ (где $k = 1,2,\ldots, N$) и способ вычисления функции ошибки E, то обучение нейронной сети превращается в задачу многомерной оптимизации, имеющую очень большую размерность, при этом, поскольку функция E может иметь произвольный вид, обучение в общем случае — многоэкстремальная невыпуклая задача оптимизации.

Способы решения задачи обучения ИНС

- Для решения этой задачи могут быть использованы следующие (итерационные) алгоритмы:
  - алгоритмы локальной оптимизации с вычислением частных производных первого порядка;
  - алгоритмы локальной оптимизации с вычислением частных производных первого и второго порядка;
  - стохастические алгоритмы оптимизации;
  - алгоритмы глобальной оптимизации.

Способы решения задачи обучения ИНС

- К первой группе относятся: градиентный алгоритм (метод скорейшего спуска); методы с одномерной и двумерной оптимизацией целевой функции в направлении антиградиента; метод сопряженных градиентов; методы, учитывающие направление антиградиента на нескольких шагах алгоритма.

Способы решения задачи обучения ИНС

- Ко второй группе относятся: метод Ньютона, методы оптимизации с разреженными матрицами Гессе, квазиньютоновские методы, метод Гаусса Ньютона, метод Левенберга-Марквардта и др.
- Данный алгоритм минимизирует комбинацию квадратов ошибок и весов с выбором наилучшей такой (для получения наилучших обобщающих свойств сети).

Способы решения задачи обучения ИНС

- Стохастическими методами являются: поиск в случайном направлении, генетический алгоритм, метод Монте-Карло (численный метод статистических испытаний).

Способы решения задачи обучения ИНС

- Задачи глобальной оптимизации решаются с помощью перебора значений переменных, от которых зависит целевая функция (функция ошибки *E)*.

Алгоритм обратного распространения

- Это итеративный градиентный алгоритм обучения, который используется с целью минимизации среднеквадратичного отклонения текущего выхода и желаемого выхода многослойных нейронных сетей.
- Алгоритм обратного распространения используется для обучения многослойных нейронных сетей с последовательными связями. Нейроны в таких сетях делятся на группы с общим входным сигналом — слои, при этом на каждый нейрон первого слоя подаются все элементы внешнего входного сигнала, а все выходы нейронов *m*-го слоя подаются на каждый нейрон слоя (q + 1). Нейроны выполняют взвешенное (с синаптическими весами) суммирование элементов входных сигналов; к данной сумме прибавляется смещение нейрона. Над полученным результатом выполняется активационной функцией затем нелинейное преобразование. Значение функции активации есть выход нейрона.

Алгоритм обратного распространения

- В многослойных сетях оптимальные выходные значения нейронов всех слоев, кроме последнего, как правило, неизвестны, и трех- или более слойный персептрон уже невозможно обучить, руководствуясь только величинами ошибок на выходах НС. Наиболее приемлемым вариантом обучения в таких условиях оказался градиентный метод поиска минимума функции ошибки с рассмотрением сигналов ошибки от выходов НС к ее входам, в направлении, обратном прямому распространению сигналов в обычном режиме работы. Этот алгоритм обучения НС получил название процедуры обратного распространения.

Алгоритм обратного распространения

- В данном алгоритме функция ошибки представляет собой сумму квадратов рассогласования (ошибки) желаемого выхода сети и реального. При вычислении элементов вектора-градиента использован своеобразный вид производных функций активации сигмоидального типа. Алгоритм действует циклически (итеративно), и его циклы принято называть эпохами. На каждой эпохе на вход сети поочередно подаются все обучающие наблюдения, выходные значения сети сравниваются с целевыми значениями и вычисляется ошибка. Значение ошибки, а также градиента поверхности ошибок используется для корректировки весов, после чего все действия повторяются. Начальная конфигурация сети выбирается случайным образом, и процесс обучения прекращается, либо когда пройдено определенное количество эпох, либо когда ошибка достигнет некоторого определенного уровня малости, либо когда ошибка перестанет уменьшаться (пользователь может сам выбрать нужное условие остановки).

Алгоритм обратного распространения

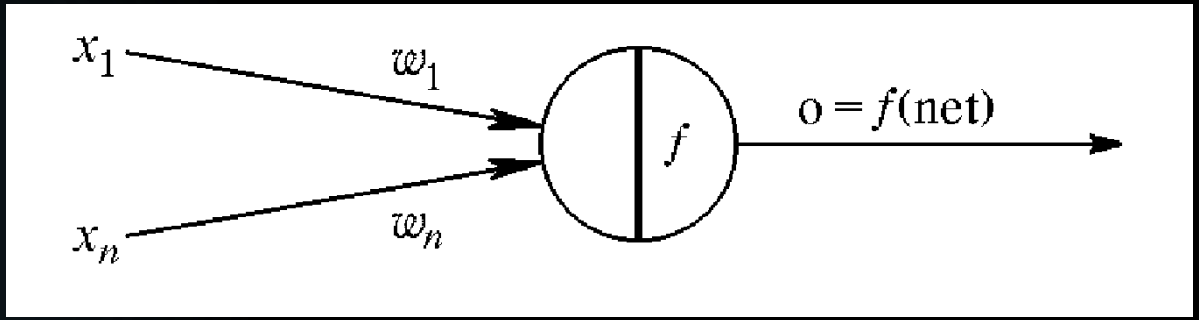
- Шаг 1. Весам сети присваиваются небольшие начальные значения.
- Шаг 2. Выбирается очередная обучающая пара **(X, Y)** из обучающего множества; вектор **X** подается на вход сети.
- Шаг 3. Вычисляется выход сети.
- Шаг 4. Вычисляется разность между требуемым (целевым, **Y**) и реальным (вычисленным) выходом сети.
- Шаг 5. Веса сети корректируются так, чтобы минимизировать ошибку.
- Шаг 6. Шаги со 2-го по 5-й повторяются для каждой пары обучающего множества до тех пор, пока ошибка на всем множестве не достигнет приемлемой величины.

Алгоритм обратного распространения

- Шаги 2 и 3 подобны тем, которые выполняются в уже обученной сети.
- Вычисления в сети выполняются послойно. На шаге 3 каждый из выходов сети вычитается из соответствующей компоненты целевого вектора с целью получения ошибки. Эта ошибка используется на шаге 5 для коррекции весов сети.
- Шаги 2 и 3 можно рассматривать как «проход вперед», так как сигнал распространяется по сети от входа к выходу. Шаги 4 и 5 составляют «обратный проход», поскольку здесь вычисляемый сигнал ошибки распространяется обратно по сети и используется для подстройки весов.

Математическое представление алгоритма

- рассмотрим вначале на примере простейшей нейронной сети, содержащей только один нейрон (условно показанный кружком):



Математическое представление алгоритма

- Будем полагать, что выход (output) сети o = *f(s)* определяется функцией активации сигмоидного типа

$$f(s) = \frac{1}{1 + e^{-as}}.$$



$$\mathrm{o} = \mathrm{o}(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}},$$

- где $x^T = (x_1, \ldots, x_n)$ — вектор входных сигналов, $w^T = (w_1, \ldots, w_n)$ — вектор весов сети, «T» — символ транспонирования.
- Предположим, далее, что для обучения сети используется выборка

$$\begin{aligned}
&\mathbf{x}^1 = (x_1^1, x_2^1, \ldots, x_n^1)^\top, \quad y^1,\\
&\mathbf{x}^2 = (x_1^2, x_2^2, \ldots, x_n^2)^\top, \quad y^2,\\
&\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\\
&\mathbf{x}^N = (x_1^N, x_2^N, \ldots, x_n^N)^\top, \quad y^N,
\end{aligned}$$

Математическое представление алгоритма

- В качестве функции ошибки для *k-го* образца *(k-го* элемента обучающей выборки) примем величину, пропорциональную квадрату разности желаемого выхода и выхода сети:

$$E_k = \frac{1}{2}(y^k - \mathrm{o}^k)^2 = \frac{1}{2}\left(y^k - \mathrm{o}^k(\mathbf{w}^\top \mathbf{x}^k)\right)^2 = \frac{1}{2}\left(y^k - \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}^k}}\right)^2.$$

- Соответственно, суммарная функция ошибки по всем элементам выборки:

$$E = \sum_{k=1}^{N} E_k.$$

Математическое представление алгоритма

- Очевидно, как E_k, так и E являются функциями вектора весов сети w. Задача обучения сети сводится в данном случае к подбору такого вектора w, при котором достигается минимум E. Данную задачу (оптимизации) будем решать градиентным методом, используя соотношение

$$\mathbf{w} := \mathbf{w} - \eta E'_k(\mathbf{w}),$$

- где «:=» — оператор присвоения, $E'_k(w)$ — обозначение вектора-градиента, η — некоторая константа.

Математическое представление алгоритма

- Представляя данный вектор в развернутом виде и учитывая выражение для
- Производной от сложной функции $f(g(s))' = f'(g(s))\cdot g'(s)$
- Производной сигмоидной функции $f'(s) = af(s)\big(1 - f(s)\big).$ где $f(s) = \dfrac{1}{1 + e^{-as}}.$
- получим: $$E'_k(\mathbf{w}) = \frac{d}{d\mathbf{w}}\left(\frac{1}{2}\left(y^k - \frac{1}{1 + e^{-\mathbf{w}^\mathsf{T}\mathbf{x}^k}}\right)^2\right) = -(y^k - \mathrm{o}^k)\mathrm{o}^k(1 - \mathrm{o}^k)\mathbf{x}^k.$$
- Это дает возможность записать алгоритм коррекции (подстройки) вектора весовых коэффициентов сети в форме $$\mathbf{w} := \mathbf{w} + \eta(y^k - \mathrm{o}^k)\mathrm{o}^k(1 - \mathrm{o}^k)\mathbf{x}^k = \mathbf{w} + \eta\delta_k\mathbf{x}^k,$$
- Где $$\delta_k = (y^k - \mathrm{o}^k)\mathrm{o}^k(1 - \mathrm{o}^k).$$

Математическое представление алгоритма

- Полученные математические выражения полностью определяют алгоритм обучения рассматриваемой НС, который может быть представлен теперь в следующем виде.
- 1. Задаются некоторые η $(0 < \eta < 1)$, E_{max} и некоторые малые случайные веса w_i сети,
- 2. Задаются $k = 1$ и $E = 0$.
- 3. Вводится очередная обучающая пара (x^k, y^k). Производятся обозначения $\mathbf{x} := \mathbf{x}^k, \quad y := y^k$
- и вычисляется величина выхода сети: $\mathrm{o} = \mathrm{o}(\mathbf{w}^\top \mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}.$

Математическое представление алгоритма

- 4. Обновляются (корректируются) веса:

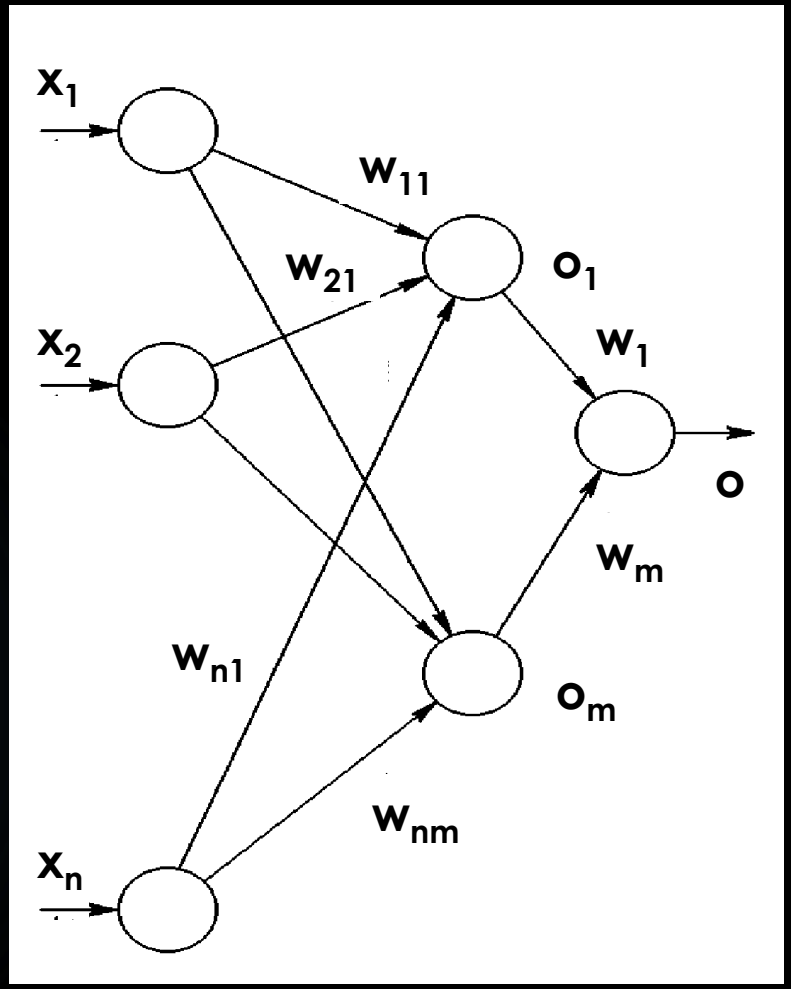
$$\mathbf{w} := \mathbf{w} + \eta(y - \mathrm{o})\mathrm{o}(1 - \mathrm{o})\mathbf{x}.$$

- 5. Корректируется (наращивается) значение функции ошибки

$$E := E + \frac{1}{2}(y - \mathrm{o})^2.$$

- 6. Если $k < N$, тогда $k := k + 1$ и переход к шагу 3, в противоположном случае — переход на шаг 7.
- 7. Завершение цикла обучения. Если $E < E_{max}$, то окончание всей процедуры обучения. Если $E \geq E_{max}$, тогда начинается новый цикл обучения переходом к шагу 2.

Обучение многослойной сети



Обучение многослойной сети

- В данном случае функция ошибки зависит от векторов весов скрытого слоя и вектора весов, связанных с выходным нейроном. Выход сети описывается выражением

$$\mathrm{O}^k = \frac{1}{1 + e^{-\mathbf{W}^\mathsf{T}\mathbf{o}^k}},$$

- где W — вектор весов выходного нейрона, o^k — вектор выходов нейронов скрытого слоя с элементами

$$\mathrm{o}_i^k = \frac{1}{1 + e^{-\mathbf{w}_i^\mathsf{T}\mathbf{x}^k}},$$

- w_i обозначает вектор весов, связанных с *i*-м скрытым нейроном, $i = 1, .., m$.

Обучение многослойной сети

- Правило корректировки весов в рассматриваемой НС также основано на минимизации квадратичной функции ошибки градиентным методом на основе выражений:

$$\mathbf{W} := \mathbf{W} - \eta \frac{\partial E_k(\mathbf{W}, \mathbf{w})}{\partial \mathbf{W}},$$

$$\mathbf{w}_i := \mathbf{w}_i - \eta \frac{\partial E_k(\mathbf{W}, \mathbf{w})}{\partial \mathbf{w}_i},$$

- где η = const — коэффициент скорости обучения ($0 < \eta < 1$), $i = 1 \ldots m$.

Обучение многослойной сети

- Используя правило дифференцирования сложной функции

$$f(g(x))' = f'(g(x))\cdot g'(x)$$

- и выражение для производной сигмоидной функции активации, получим:

$$\frac{\partial E_k(\mathbf{W}, \mathbf{w})}{\partial \mathbf{W}} = \frac{1}{2}\frac{\partial}{\partial \mathbf{W}}\left(y^k - \frac{1}{1+e^{-\mathbf{W}^\top \mathbf{o}^k}}\right)^2 = \\ = -(y^k - \mathrm{O}^k)\mathrm{O}^k(1-\mathrm{O}^k)\mathbf{o}^k,$$

- откуда следует

$$\mathbf{W} := \mathbf{W} + \eta(y^k - \mathrm{O}^k)\mathrm{O}^k(1-\mathrm{O}^k)\mathbf{o}^k = \mathbf{W} + \eta\delta_k\mathbf{o}^k,$$

- или в скалярной форме $W_i := W_i + \eta\delta_k \mathrm{o}_i^k,$ *i = 1, .., m.*

- Где $\delta_k = (y^k - \mathrm{O}^k)\mathrm{O}^k(1-\mathrm{O}^k).$

Обучение многослойной сети

- Поступая аналогично, найдем

$$\frac{\partial E_k(\mathbf{W}, \mathbf{w})}{\partial \mathbf{w}_i} = -(y^k - \mathrm{O}^k)\mathrm{O}^k(1 - \mathrm{O}^k)W_i \mathrm{o}_i^k(1 - \mathrm{o}_i^k)\mathbf{x}^k,$$

- откуда получаем

$$\mathbf{w}_i := \mathbf{w}_i + \eta\delta_k W_i \mathrm{o}_i^k(1 - \mathrm{o}_i^k)\mathbf{x}^k,$$

- или (в скалярной форме)

$$\mathbf{w}_{ij} := \mathbf{w}_{ij} + \eta\delta_k W_i \mathrm{o}_i^k(1 - \mathrm{o}_i^k)x_j^k, \quad i = 1, 2, \ldots, m \quad j = 1, 2, \ldots, n.$$

Обучение многослойной сети

- Алгоритм обучения может быть теперь представлен в виде следующих шагов.
- 1. Задаются некоторые η ($0 < \eta < 1$), E_{max} и некоторые малые случайные веса w_i сети.
- 2. Задаются k=1 и E = 0.
- 3. Вводится очередная обучающая пара (x^k, y^k). Производятся обозначения x := x^k, y := y^k и вычисляется величина выхода сети:

$$O = \frac{1}{1 + e^{-\mathbf{W}^T \mathbf{o}}},$$

- где **W** — вектор весов выходного нейрона, $\mathbf{o}^k$ — вектор выходов нейронов скрытого слоя с элементами

Обучение многослойной сети

$$o_i = \frac{1}{1 + e^{-\mathbf{w}_i^T \mathbf{x}}},$$

- w_i обозначает вектор весов, связанных с *i*-м скрытым нейроном, $i = 1,2,\ldots, m$.
- 4. Производится корректировка весов выходного нейрона:

$$\mathbf{W} := \mathbf{W} + \eta\delta\mathbf{o},$$

- Где

$$\delta = (y - O)O(1 - O)$$

- 5. Корректируются веса нейронов скрытого слоя:

$$\mathbf{w}_i := \mathbf{w}_i + \eta\delta W_i o_i(1 - o_i)\mathbf{x}, \quad i = 1, 2, \ldots, m$$

Обучение многослойной сети

- 6. Корректируется (наращивается) значение функции ошибки:

$$E := E + \frac{1}{2}(y - \mathrm{o})^2.$$

- Если $k < N$, тогда $k := k + 1$ и переход к шагу 3, в противоположном случае переход на шаг 8.
- 7. Завершение цикла обучения. Если $E < E_{max}$, то окончание всей процедуры обучения. Если $E \geq E_{max}$, тогда начинается новый цикл обучения переходом к шагу 2.

Обучение многослойной сети

- Рассмотренная процедура может быть легко обобщена на случай сети с произвольным количеством слоев и нейронов в каждом слое. Обратим внимание, что в данной процедуре сначала происходит коррекция весов для выходного нейрона, а затем — для нейронов скрытого слоя, т.е. от конца сети к ее началу. Отсюда и название — обратное распространение ошибки. Ввиду использования для обозначений греческой буквы δ, эту процедуру обучения называют еще иногда *обобщенным дельта-правилом*.

Геометрическая интерпретация градиентного метода

- В алгоритме *обратного распространения* вычисляется вектор градиента поверхности ошибок. Этот вектор указывает направление кратчайшего спуска по поверхности из данной точки, поэтому, если мы «немного» продвинемся по нему, ошибка уменьшится. Последовательность таких шагов (замедляющаяся по мере приближения к дну) в конце концов приведет к минимуму того или иного типа. Определенную трудность здесь представляет вопрос о том, какую нужно брать длину шагов (что определяется величиной коэффициента скорости обучения η).

Геометрическая интерпретация градиентного метода

- При большой длине шага сходимость будет более быстрой, но имеется опасность перепрыгнуть через решение или (если поверхность ошибок имеет особо вычурную форму) уйти в неправильном направлении. Классическим примером такого явления при обучении нейронной сети является ситуация, когда алгоритм очень медленно продвигается по узкому оврагу с крутыми склонами, прыгая с одной его стороны на другую. Напротив, при маленьком шаге, вероятно, будет схвачено верное направление, однако при этом потребуется очень много итераций. На практике величина шага берется пропорциональной крутизне склона (так что алгоритм замедляет ход вблизи минимума) с некоторой константой (η), которая, как отмечалось, называется коэффициентом скорости обучения. Правильный выбор скорости обучения зависит от конкретной задачи и обычно осуществляется опытным путем; эта константа может также зависеть от времени, уменьшаясь по мере продвижения алгоритма.

Анализ работы алгоритма

- Обычно этот алгоритм видоизменяется таким образом, чтобы включать слагаемое импульса (или инерции). Этот член способствует продвижению в фиксированном направлении, поэтому если было сделано несколько шагов в одном и том же направлении, то алгоритм «увеличивает скорость», что (иногда) позволяет избежать локального минимума, а также быстрее проходить плоские участки.
- Таким образом, алгоритм действует итеративно, и его шаги принято называть эпохами. На каждой эпохе на вход сети поочередно подаются все обучающие наблюдения, выходные значения сети сравниваются с целевыми значениями и вычисляется ошибка. Значение ошибки, а также градиента поверхности ошибок используется для корректировки весов, после чего все действия повторяются. Начальная конфигурация сети выбирается случайным образом, и процесс обучения прекращается, либо когда пройдено определенное количество эпох, либо когда ошибка достигнет некоторого определенного уровня малости, либо когда ошибка перестанет уменьшаться (пользователь может сам выбрать нужное условие остановки).

Анализ работы алгоритма

- Классический метод обратного распространения относится к алгоритмам с линейной сходимостью и известными недостатками его являются: невысокая скорость сходимости (большое число требуемых итераций для достижения минимума функции ошибки), возможность сходиться не к глобальному, а к локальным решениям (локальным минимумам отмеченной функции), возможность паралича сети (при котором большинство нейронов функционирует при очень больших значениях аргумента функций активации, т. е. на ее пологом участке; поскольку ошибка пропорциональна производной, которая на данных участках мала, то процесс обучения практически замирает).
- Для устранения этих недостатков были предложены многочисленные модификации алгоритма обратного распространения, которые связаны с использованием различных функций ошибки, различных процедур определения направления и величины шага и т. п.

Переобучение и обобщение

- Одна из наиболее серьезных трудностей изложенного подхода заключается в том, что таким образом мы минимизируем не ту ошибку, которую на самом деле нужно минимизировать, — ошибку, которую можно ожидать от сети, когда ей будут подаваться совершенно новые наблюдения.
- Иначе говоря, мы хотели бы, чтобы нейронная сеть обладала способностью *обобщать* результат на новые наблюдения. В действительности сеть обучается минимизировать ошибку на обучающем множестве, и в отсутствие идеального и бесконечно большого обучающего множества это совсем не то же самое, что минимизировать «настоящую» ошибку на поверхности ошибок в заранее неизвестной модели явления.

Переобучение и обобщение

- Сильнее всего это различие проявляется в проблеме переобучения, или слишком близкой подгонки. Это явление проще будет продемонстрировать не для нейронной сети, а на примере аппроксимации посредством полиномов, — при этом суть явления абсолютно та же.
- Полином (или многочлен) — это выражение, содержащее только константы и целые степени независимой переменной. Примеры: $y = 2x + 3, \quad y = 3x^2 + 4x + 1.$

Переобучение и обобщение

- Графики полиномов могут иметь различную форму, причем чем выше степень многочлена (и, тем самым, чем больше членов в него входит), тем более сложной может быть эта форма. Если у нас есть некоторые данные, мы можем поставить цель подогнать к ним полиномиальную кривую (модель) и получить таким образом объяснение для имеющейся зависимости. Наши данные могут быть зашумлены, поэтому нельзя считать, что самая лучшая модель задается кривой, которая в точности проходит через все имеющиеся точки. Полином низкого порядка может быть недостаточно гибким средством для аппроксимации данных, в то время как полином высокого порядка может оказаться чересчур гибким, и будет точно следовать данным, принимая при этом замысловатую форму, не имеющую никакого отношения к форме настоящей зависимости.

Переобучение и обобщение

- Нейронная сеть сталкивается с точно такой же трудностью. Сети с большим числом весов моделируют более сложные функции и, следовательно, склонны к переобучению. Сеть же с небольшим числом весов может оказаться недостаточно гибкой, чтобы смоделировать имеющуюся зависимость.
- Как же выбрать «правильную» степень сложности для сети? Почти всегда более сложная сеть дает меньшую ошибку, но это может свидетельствовать не о хорошем качестве модели, а о переобучении.

Переобучение и обобщение

- Ответ состоит в том, чтобы использовать механизм контрольной кросс-проверки, при котором часть обучающих наблюдений резервируется и в обучении по алгоритму обратного распространения не используется. Вместо этого, по мере работы алгоритма, она используется для независимого контроля результата. В самом начале работы ошибки сети на обучающем и контрольном множествах будут одинаковыми (если они существенно отличаются, то, вероятно, разбиение всех наблюдений на два множества было неоднородно). По мере того как сеть обучается, ошибка обучения, естественно, убывает, и, пока обучение уменьшает действительную функцию ошибок, ошибка на контрольном множестве также будет убывать. Если же контрольная ошибка перестала убывать или даже стала расти, это указывает на то, что сеть начала слишком близко аппроксимировать данные и обучение следует остановить. Это явление чересчур точной аппроксимации в процессе обучения и называется переобучением. Если такое случилось, то обычно советуют уменьшить число скрытых элементов и/или слоев, ибо сеть является слишком мощной для данной задачи. Если же сеть, наоборот, была взята недостаточно богатой для того, чтобы моделировать имеющуюся зависимость, то переобучения, скорее всего, не произойдет, и обе ошибки — обучения и проверки — не достигнут достаточного уровня малости.

- Описанные проблемы с локальными минимумами и выбором размера сети приводят к тому, что при практической работе с нейронными сетями, как правило, приходится экспериментировать с большим числом различных сетей, порой обучая каждую из них по несколько раз (чтобы не быть введенным в заблуждение локальными минимумами) и сравнивая полученные результаты. Главным показателем качества результата является здесь контрольная ошибка. При этом в соответствии с общенаучным принципом, согласно которому при прочих равных следует предпочесть более простую модель, из двух сетей с приблизительно равными ошибками контроля имеет смысл выбрать ту, которая меньше.

Модификации алгоритма

- Классический метод обратного распространения относится к алгоритмам с линейной сходимостью. Для увеличения скорости сходимости необходимо использовать матрицы вторых производных функции ошибки.
- Были предложены многочисленные модификации алгоритма обратного распространения, которые связаны с использованием различных функций ошибки, различных процедур определения направления и величины шага.
- 1. Функции ошибки:
    - интегральные функций ошибки по всей совокупности обучающих примеров;
    - функции ошибки целых и дробных степеней.
- 2. Процедуры определения величины шага на каждой итерации:
    - метод бисекции;
    - инерционные соотношения.
- 3. Процедуры определения направления шага:
    - с использованием матрицы производных второго порядка (метод Ньютона и др.);
    - с использованием направлений на нескольких шагах.